

Fiche 316 — Calcul sur le langage : `quote`, `substitute`, `eval`, `match.call`

Matière	Code · Langage R
Cours source	R Core Team, <i>R Language Definition</i> 4.6.1 — chapitre 6 « Computing on the language » (§6.1 manipulation directe, §6.2 substitutions, §6.3 évaluation, §6.4 objets expression, §6.5 manipulation des appels, §6.6 manipulation des fonctions)
Sources d'appoint	<i>R Language Definition</i> §1 « Introduction », §2.1.3 « Language objects », §2.1.4 « Expression objects », §2.1.8 « Promise objects »
Difficulté	Avancé — la partie la plus « Lisp » de R
Temps d'étude estimé	1 h 45
Prérequis	Fiches 303, 306, 309, 310 (types, listes, promesses, environnements)
Concepts clés	objet <code>call</code> , <code>quote()</code> , accès façon liste , <code>as.list</code> / <code>as.call</code> , <code>as.name</code> , parenthèses conservées , <code>deparse()</code> , <code>substitute()</code> et les promesses , substitution purement lexicale , <code>bquote()</code> et <code>.(())</code> , <code>eval()</code> et son environnement, <code>eval.parent</code> , objets expression , <code>sys.call()</code> , <code>match.call()</code> , <code>expand.dots</code> , <code>call()</code> , <code>do.call()</code> , <code>body</code> / <code>formals</code> / <code>environment</code>
À retenir en priorité	Un appel est une liste dont le premier élément est la fonction · <code>substitute</code> lit la fente expression d'une promesse · la substitution est purement lexicale · <code>match.call()</code> puis <code>mf[[1]]</code> <code><- as.name(...)</code> · <code>do.call</code> .

Vue d'ensemble

LE PRINCIPE « R belongs to a class of programming languages in which SUBROUTINES HAVE THE ABILITY TO MODIFY OR CONSTRUCT OTHER SUBROUTINES and EVALUATE THE RESULT as an integral part of the language itself. »
« the engine is really VERY LISP-LIKE »

TROIS OBJETS DE LANGAGE les CALLS, les EXPRESSIONS, les FONCTIONS

```

CAPTURER      e1 <- quote(2 + 2)          e2 <- quote(plot(x, y))
              « The arguments are NOT EVALUATED, the result is simply
              the PARSED ARGUMENT. »

UN APPEL EST UNE LISTE
e2[[1]] plot    <- la FONCTION
e2[[2]] x       <- le 1er argument
e2[[3]] y
e3$x           <- si l'argument etait NOMME
as.list / as.call pour convertir dans les deux sens
as.name("+") pour fabriquer un symbole

SUBSTITUER    substitute(x) dans une fonction -> L'EXPRESSION PASSEE
deparse(substitute(x)) -> son TEXTE (etiquettes d'axes !)
substitute(a + b, list(a = 1, b = quote(x))) -> 1 + x
!! « the substitutions are PURELY LEXICAL ; there is NO CHECKING
    that the resulting call objects MAKE SENSE »
bquote( x[.(i)] == .(pnorm(i)) ) -- tout est quote SAUF .()

EVALUER       eval(e)                  dans le cadre courant
              eval.parent(e)           = eval(expr, sys.frame(sys.parent()))
              eval(expr, data, enclos)  dans une liste, avec un ENCLOS

L'APPEL COURANT sys.call() tel qu'il a ete ecrit
                match.call() AVEC LES ARGUMENTS APPARIES <- le bon outil
                mf$arg <- NULL        retirer un argument
                mf$arg <- valeur      en ajouter un
                mf[[1]] <- as.name("model.frame") CHANGER LA FONCTION

CONSTRUIRE    call("round", x) -- la VALEUR de x est inseree
              do.call(cbind, lapply(x, is.na)) -- construit ET evalue

DISSEQUER UNE FONCTION  body() formals() environment() et leurs <-

```

Le problème posé. « *R belongs to a class of programming languages in which subroutines have the ability to modify or construct other subroutines and evaluate the result as an integral part of the language itself. This is similar to Lisp and Scheme ... but in contrast to FORTRAN and the ALGOL family.* » (§6)

INTUITION — LA PHRASE QUI SITUE R.

« *R presents a friendlier interface to programming than Lisp does, at least to someone used to mathematical formulas and C-like control structures, **but the engine is really very Lisp-like. R allows direct access to parsed expressions and functions and allows you to alter and subsequently execute them, or create entirely new functions from scratch.*** »

L'introduction du manuel (§1) le dit autrement : « *the semantics are of the FPL (functional programming language) variety **with stronger affinities with Lisp and APL.** In particular, **it allows "computing on the language", which in turn makes it possible to write functions that take expressions as input, something that is often useful for statistical modeling and graphics.*** »

À quoi cela sert (§6). *« There is a number of **standard applications** of this facility, such as calculation of analytical derivatives of expressions, or the generation of polynomial functions from a vector of coefficients. However, there are also uses that are much more fundamental to the workings of the interpreted part of R. Some of these are essential to the reuse of functions as components in other functions, as the (admittedly not very pretty) calls to `model.frame` that are constructed in several modeling and plotting routines. **Other uses simply allow elegant interfaces to useful functionality** — as an example, consider the `curve` function, which allows you to **draw the graph of a function given as an expression like `sin(x)`**. »*

Ce chapitre explique donc trois choses déjà rencontrées : pourquoi `plot(x, y)` sait écrire « x » et « y » sur les axes (fiche 314), comment `lm(y ~ x, data = df)` évalue sa formule chez l'appelant (fiche 313), et ce qu'est vraiment la **portée dynamique simulée** (fiche 310).

● Concept 1 — Capturer une expression

DÉFINITION (§6.1).

« *There are three kinds of language objects that are available for modification : calls, expressions, and functions. At this point, we shall concentrate on the **call** objects. These are sometimes referred to as "unevaluated expressions", although this terminology is somewhat confusing. The most direct method of obtaining a call object is to use `quote` with an expression argument.* »

```
e1 <- quote(2 + 2)
e2 <- quote(plot(x, y))
```

*« **The arguments are not evaluated, the result is simply the parsed argument.** The objects `e1` and `e2` may be **evaluated later using** `eval`, or **simply manipulated as data.** »*

La révélation du §6.1. « *It is perhaps most immediately obvious why the `e2` object has mode "call", since it involves a call to the `plot` function. However, `e1` actually has exactly the same structure as a call to the binary operator `+` with two arguments* » :

```
> quote("+"(2, 2))
2 + 2
```

`2 + 2` **est un appel de fonction.** C'est la mise en pratique de la règle de la fiche 301 : « *except for the syntax, there is no difference between applying an operator and calling a function* ».

● Concept 2 — Un appel est une liste

Règle (§6.1). « *The components of a call object are accessed using a list-like syntax, and may in fact be converted to and from lists using `as.list` and `as.call`.* »

```
> e2[[1]]
plot
> e2[[2]]
x
> e2[[3]]
y
```

Le premier élément est la fonction, les suivants sont les arguments. C'est exactement la structure d'une liste (fiche 306), avec `[[` pour l'accès.

Règle — les arguments nommés (§6.1). « *When keyword argument matching is used, the keywords can be used as list tags.* »

```
> e3 <- quote(plot(x = age, y = weight))
> e3$x
age
> e3$y
weight
```

Règle — de quoi sont faits les composants (§6.1). « *All the components of the call object have mode "name" in the preceding examples. This is true for identifiers in calls, but the components of a call can also be constants — which can be of any type, although the first component had better be a function if the call is to be evaluated successfully — or other call objects, corresponding to subexpressions.* »

« *Objects of mode name can be constructed from character strings using as.name* » :

```
> e2[[1]] <- as.name("+")
> e2
x + y
```

On vient de transformer un appel à `plot` en une addition, en changeant un seul élément.

Et l'imbrication (§6.1). « *To illustrate the fact that subexpressions are simply components that are themselves calls, consider* » :

```
> e1[[2]] <- e2
> e1
x + y + 2
```

CONTRÔLE — VÉRIFIER CE RÉSULTAT.

`e1` était `quote(2 + 2)`, c'est-à-dire l'appel ``+`(2, 2)`. Remplacer son **deuxième** élément — le premier opérande, `2` — par `e2`, devenu `x + y`, donne ``+`(x + y, 2)`, que le dépareur écrit `x + y + 2`.

L'arbre syntaxique est un objet R ordinaire, modifiable à la pince.

2.1 Les parenthèses, et l'inversibilité imparfaite

Règle (§6.1). « *All grouping parentheses in input are preserved in parsed expressions. They are represented as a function call with one argument, so that $4 - (2 - 2)$ becomes `"-(4, "( " (-(2, 2))` in prefix notation. In evaluations, the `(` operator just returns its argument.* »

« *This is a bit unfortunate, but it is not easy to write a parser/deparsed combination that both preserves user input, stores it in minimal form and ensures that parsing a deparsed expression gives the same expression back.* »

Règle (§6.1). « *As it happens, R's parser is not perfectly invertible, nor is its deparser, as the following examples show* » :

```
> str(quote(c(1,2)))
language c(1, 2)
> str(c(1,2))
num [1:2] 1 2

> deparse(quote(c(1,2)))
[1] "c(1, 2)"
> deparse(c(1,2))
[1] "c(1, 2)"

> quote("-(2, 2))
2 - 2
> quote(2 - 2)
2 - 2
```

Deux objets radicalement différents — un appel et un vecteur numérique — ont le même dépareillage. Et deux écritures différentes se déparent identiquement.

« *Deparsed expressions should, however, evaluate to an equivalent value to the original expression (up to rounding error).* »

EN CLAIR.

Le dépareillage est **une représentation, pas une identité**. On ne peut pas conclure de `deparse(a) == deparse(b)` que `a` et `b` sont le même objet. Pour comparer des expressions, `identical()` est le bon outil.

● Concept 3 — `substitute()`

L'exemple qui motive tout (§6.2). « *It is in fact not often that one wants to modify the innards of an expression like in the previous section. More frequently, one wants to simply get at an expression in order to deparse it and use it for labeling plots. An example of this is seen at the beginning of `plot.default`* » :

```
xlabel <- if (!missing(x))
  deparse(substitute(x))
```

« *This causes the variable or expression given as the `x` argument to `plot` to be used for labeling the x-axis later on.* »

Voilà pourquoi `plot(poids, taille)` étiquette les axes « **poids** » et « **taille** ». La fonction ne reçoit pas ces noms : elle **les retrouve**.

DÉFINITION (§6.2).

*« The function used to achieve this is `substitute`, which **takes the expression `x` and substitutes the expression that was passed through the formal argument `x`**. Notice that for this to happen, `x` must carry information about the expression that creates its value. This is related to the lazy evaluation scheme of R. A formal argument is really a promise, an object with three slots — one for **the expression that defines it**, one for **the environment in which to evaluate that expression**, and one for **the value** once evaluated. `substitute` will recognize a promise variable and substitute the value of its expression slot. If `substitute` is invoked inside a function, the local variables of the function are also subject to substitution. »*

C'est le mécanisme des promesses de la fiche 309, vu du côté de l'utilisateur.

3.1 Substituer dans une expression quelconque

Règle (§6.2). « *The argument to `substitute` does not have to be a simple identifier, it can be an expression involving several variables and substitution will occur for each of these. Also, `substitute` has an additional argument which can be an environment or a list in which the variables are looked up.* »

```
> substitute(a + b, list(a = 1, b = quote(x)))  
1 + x
```

« **Notice that quoting was necessary to substitute the `x`.** » Sans `quote(x)`, R chercherait la valeur de `x` ; avec, il substitue le symbole.

L'usage en graphique (§6.2) — « *This kind of construction comes in handy in connection with the facilities for putting math expression in graphs* » :

```
plot(0)  
for (i in 1:4)  
  text(1, 0.2 * i,  
       substitute(x[ix] == y, list(ix = i, y = pnorm(i))))
```

3.2 Purement lexical

Règle (§6.2). « *It is important to realize that the substitutions are purely lexical ; there is no checking that the resulting call objects make sense if they are evaluated.*

`substitute(x <- x + 1, list(x = 2))` will happily return `2 <- 2 + 1`. »

« However, **some parts of R make up their own rules for what makes sense and what does not and might actually have a use for such ill-formed expressions.** For example, using the "math in graphs" feature often involves constructions that are syntactically correct, but which would be meaningless to evaluate, like `{ }>=40*" years"`. »

Ce n'est pas un défaut mais une propriété : `substitute` construit **de la syntaxe**, pas du sens. Et l'annotation mathématique (fiche 314) **exploite** cette liberté — une expression destinée à être **dessinée** n'a pas à être évaluable.

3.3 L'énigme de la double substitution

PROBLÈME (§6.2).

« `substitute` *will not evaluate its first argument. This leads to the puzzle of how to do substitutions on an object that is contained in a variable. The solution is to use `substitute` once more* » :

```
> expr <- quote(x + y)
> substitute(substitute(e, list(x = 3)), list(e = expr))
substitute(x + y, list(x = 3))
> eval(substitute(substitute(e, list(x = 3)), list(e = expr)))
3 + y
```

COMMENT LIRE CETTE CONSTRUCTION.

Le `substitute` **extérieur** remplace `e` par **le contenu de `expr`** — il produit donc l'appel `substitute(x + y, list(x = 3))`, non encore évalué. Le `eval` déclenche alors ce second `substitute`, qui remplace `x` par `3`. **Deux substitutions en cascade**, la première fabriquant l'appel que la seconde exécute.

3.4 Les règles exactes

Règle (§6.2). « *The exact rules for substitutions are as follows : each symbol in the parse tree for the first is matched against the second argument, which can be a tagged list or an environment frame.* »

Cas	Ce qui se passe
un objet local simple	« <i>its value is inserted, except if matching against the global environment</i> »
une promesse (typiquement un argument)	« <i>the promise expression is substituted</i> »
le symbole n'est pas apparié	« <i>it is left untouched</i> »

« The special exception for substituting at the top level is admittedly peculiar. It has been inherited from S and the rationale is most likely that there is no control over which variables might be bound at that level so that it would be better to just make `substitute` act as `quote .` »

Conséquence pratique : `substitute(x)` tapé à la console rend `x`, pas sa valeur. Le mécanisme ne fonctionne que dans une fonction.

3.5 Le piège de l'évaluation paresseuse

PROBLÈME (§6.2).

« The rule of promise substitution is slightly different from that of S if the local variable is modified before `substitute` is used. R will then use the new value of the variable, whereas S will unconditionally use the argument expression — unless it was a constant, which has the curious consequence that `f((1))` may be very different from `f(1)` in S. The R rule is considerably cleaner, although it does have consequences in connection with lazy evaluation that comes as a surprise to some. »

```
logplot <- function(y, ylab = deparse(substitute(y))) {
  y <- log(y)
  plot(y, ylab = ylab)
}
```

« *This looks straightforward, but one will discover that the y label becomes an ugly `c(...)` expression. It happens because the rules of lazy evaluation cause the evaluation of the `yLab` expression to happen after `y` has been modified. The solution is to force `yLab` to be evaluated first* » :

```
logplot <- function(y, ylab = deparse(substitute(y))) {  
  ylab          # <- FORCE la promesse AVANT la modification  
  y <- log(y)  
  plot(y, ylab = ylab)  
}
```

Et l'erreur à ne pas commettre (§6.2). « *Notice that one should not use `eval(yLab)` in this situation. If `yLab` is a language or expression object, then that would cause the object to be evaluated as well, which would not at all be desirable if a math expression like `quote(log[e](y))` was being passed.* »

Mentionner l'argument suffit à forcer la promesse ; `eval()` irait **un cran trop loin** et détruirait une étiquette mathématique. C'est l'idiome du forçage déjà rencontré en fiche 309 avec `label = deparse(x)` .

3.6 `bquote()` — la variante compacte

DÉFINITION (§6.2).

« *A variant on `substitute` is `bquote`, which is used to replace some subexpressions with their values.* » L'exemple du §3.1 s'écrit alors plus brièvement :

```
plot(0)  
for (i in 1:4)  
  text(1, 0.2*i, bquote( x[.(i)] == .(pnorm(i)) ))
```

« *The expression is quoted except for the contents of `.()` subexpressions, which are replaced [par leur valeur].* »

La différence de lecture : `substitute` demande **une liste de correspondances** ; `bquote` **marque les trous directement dans l'expression**, par `()` . Sur une expression longue avec deux ou trois substitutions, `bquote` est nettement plus lisible.

● Concept 4 — `eval()` et son environnement

Règle (§6.3). « The `eval` function was introduced earlier ... **However, this is not the full story. It is also possible to specify the environment in which the evaluation is to take place. By default this is the evaluation frame from which `eval` is called, but quite frequently it needs to be set to something else.** »

Cas 1 — le cadre de l'appelant (§6.3). « **Very often, the relevant evaluation frame is that of the parent of the current frame. In particular, when the object to evaluate is the result of a `substitute` operation of the function arguments, it will contain variables that make sense to the caller only** (notice that **there is no reason to expect that the variables of the caller are in the lexical scope of the callee**). **Since evaluation in the parent frame occurs frequently, an `eval.parent` function exists as a shorthand for `eval(expr, sys.frame(sys.parent()))`.** »

La parenthèse est décisive et renvoie à la fiche 310 : la portée est **lexicale**, donc les variables de l'appelant ne sont **pas** visibles depuis la fonction appelée. Évaluer explicitement dans le cadre parent est **la simulation de portée dynamique** dont parlait le §3.5.3.

Cas 2 — une liste ou un data frame (§6.3). « Another case that occurs frequently is **evaluation in a list or a data frame**. For instance, this happens **in connection with the `model.frame` function when a `data` argument is given. Generally, the terms of the model formula need to be evaluated in `data`** , but they may occasionally also contain references to items in the caller of `model.frame` So for this purpose **one needs not only to evaluate an expression in a list, but also to specify an enclosure into which the search continues if the variable is not in the list.** »

```
eval(expr, data, sys.frame(sys.parent()))
```

C'est l'explication technique du `data = de lm()` (fiche 313) : « any variables needed to construct the model should **come first** from the data frame » — et **ensuite** chez l'appelant. Les trois arguments d' `eval` sont exactement cette phrase.

Règle (§6.3). « *Notice that evaluation in a given environment may actually change that environment, most obviously in cases involving the assignment operator* » :

```
eval(quote(total <- 0), environment(robert$balance)) # rob Rob
```

« *This is also true when evaluating in lists, but the original list does not change because one is really working on a copy.* »

La plaisanterie du manuel — « **rob Rob** » — **est instructive** : elle vide le compte en banque de la fiche 310 **de l'extérieur**, en évaluant une assignation dans l'environnement capturé par la fermeture. Les environnements **ne sont pas copiés** (fiche 303) ; **les listes le sont**.

● Concept 5 — Les objets `expression`

DÉFINITION (§6.4).

« *Objects of mode "expression" ... are very similar to lists of call objects.* »

```
> ex <- expression(2 + 2, 3 + 4)
> ex[[1]]
2 + 2
> ex[[2]]
3 + 4
> eval(ex)
[1] 7
```

*« **Notice that evaluating an expression object evaluates each call in turn, but the final value is that of the last call.** In this respect **it behaves almost identically to the compound language object** `quote({2 + 2; 3 + 4})` . »*

La différence subtile (§6.4). « *Call objects are indistinguishable from subexpressions in a parse tree. This means that they are automatically evaluated in the same way a subexpression would be. Expression objects can be recognized during evaluation and in a sense retain their quotedness. The evaluator will not evaluate an expression object recursively, only when it is passed directly to `eval`.* »

```
> eval(substitute(mode(x), list(x = quote(2 + 2))))
[1] "numeric"
> eval(substitute(mode(x), list(x = expression(2 + 2))))
[1] "expression"
```

Le premier a été évalué en chemin, le second non. Un objet `expression` **résiste** à l'évaluation ; un `call` **s'y fond**.

La confusion du débogueur (§6.4). « *The debugger represents an expression object by the call that creates it. ... However, it does lead to the following bit of confusion* » :

```
> e <- quote(expression(2 + 2)); e           # expression(2 + 2)
> mode(e)                                   # [1] "call"
> ee <- expression(2 + 2); ee               # expression(2 + 2)
> mode(ee)                                 # [1] "expression"
```

« *i.e., `e` and `ee` look identical when printed, but one is a call that generates an expression object and the other is the object itself.* » — **une raison de plus de ne jamais se fier à l'affichage** pour identifier un objet de langage (concept 2.1).

● Concept 6 — Manipuler l'appel courant

DÉFINITION (§6.5).

« *It is possible for a function to find out how it has been called by looking at the result of `sys.call`* » :

```
> f <- function(x, y, ...) sys.call()
> f(y = 1, 2, z = 3, 4)
f(y = 1, 2, z = 3, 4)
```

« *However, this is not really useful except for debugging because it requires the function to keep track of argument matching in order to interpret the call. For instance, it must be able to see that the 2nd actual argument gets matched to the first formal one.* »

DÉFINITION (§6.5).

« *More often one requires the call with all actual arguments bound to the corresponding formals. To this end, the function `match.call` is used.* »

```
> f <- function(x, y, ...) match.call()
> f(y = 1, 2, z = 3, 4)
f(x = 2, y = 1, z = 3, 4)
```

« *Notice that the second argument now gets matched to `x` and appears in the corresponding position in the result.* » — l'appariement en trois passes de la fiche 309, **rendu visible**.

6.1 Le patron de `lm`

MÉTHODE (§6.5).

« *The primary use of this technique is to call another function with the same arguments, possibly deleting some and adding others. A typical application is seen at the start of the `lm` function* » :

```
mf <- cl <- match.call()
mf$singular.ok <- mf$model <- mf$method <- NULL
mf$x <- mf$y <- mf$qr <- mf$contrasts <- NULL
mf$drop.unused.levels <- TRUE
mf[[1]] <- as.name("model.frame")
mf <- eval(mf, sys.frame(sys.parent()))
```

Décodage, ligne par ligne (§6.5). « *The call can be treated as a list object where the first element is the name of the function and the remaining elements are the actual argument expressions, with the corresponding formal argument names as tags.* »

Ligne	Ce qu'elle fait
1	capture l'appel avec arguments appariés , en deux copies (<code>mf</code> à transformer, <code>cl</code> à conserver)
2-3	« <i>the technique to eliminate undesired arguments is to assign</i> <code>NULL</code> »
4	« <i>to add an argument one uses tagged list assignment</i> »
5	« <i>To change the name of the function called, assign to the first element of the list and make sure that the value is a name</i> — either using <code>as.name("model.frame")</code> or <code>quote(model.frame)</code> »
6	« <i>the resulting call is evaluated in the parent frame, in which one can be certain that the involved expressions make sense</i> »

`lm` **transforme son propre appel en un appel à `model.frame`**, puis l'exécute chez l'appelant. C'est le « *admittedly not very pretty* » annoncé au §6 — et c'est ce qui fait fonctionner `lm(y ~ x, data = df)`.

6.2 `expand.dots`

Règle (§6.5). « The `match.call` function has an `expand.dots` argument, a switch which if set to `FALSE` lets all `...` arguments be collected as a single argument with the tag `...` . »

```
> f <- function(x, y, ...) match.call(expand.dots = FALSE)
> f(y = 1, 2, z = 3, 4)
f(x = 2, y = 1, ... = list(z = 3, 4))
```

« The `...` argument is a list (a pairlist to be precise), not a call to `list` like it is in `S` » :

```
> e1 <- f(y = 1, 2, z = 3, 4)$...
> e1
$z
[1] 3

[[2]]
[1] 4
```

Deux usages, du plus simple au plus élaboré (§6.5).

1. Se débarrasser des `...` . « *One reason for using this form of `match.call` is simply to get rid of any `...` arguments in order not to be passing unspecified arguments on to functions that may not know them.* » — paraphrasé de `plot.formula` :

```
m <- match.call(expand.dots = FALSE)
m$... <- NULL
m[[1]] <- "model.frame"
```

2. Fusionner des arguments. « *A more elaborate application is in `update.default` where a set of optional extra arguments can add to, replace, or cancel those of the original call* » :

```
extras <- match.call(expand.dots = FALSE)$...
if (length(extras) > 0) {
  existing <- !is.na(match(names(extras), names(call)))
  for (a in names(extras)[existing]) call[[a]] <- extras[[a]]
  if (any(!existing)) {
    call <- c(as.list(call), extras[!existing])
    call <- as.call(call)
  }
}
```

« *Notice that care is taken to modify existing arguments individually in case `extras[[a]] == NULL` . Concatenation does not work on call objects without the coercion as shown ; this is arguably a bug.* »

C'est le moteur d' `update()` (fiche 313) : les arguments existants sont **remplacés un par un**, les nouveaux **concaténés** après passage par `as.list` puis `as.call` .

6.3 Construire un appel

DÉFINITION (§6.5).

« *Two further functions exist for the construction of function calls, namely `call` and `do.call`.* »

`call` « *allows creation of a call object **from the function name and the list of arguments*** » :

```
> x <- 10.5
> call("round", x)
round(10.5)
```

« *As seen, the value of `x` rather than the symbol is inserted in the call, so it is distinctly different from `round(x)`. The form is used rather rarely, but is occasionally useful where the name of a function is available as a character variable.* »

`do.call` « *is related, but **evaluates the call immediately** and **takes the arguments from an object of mode "list"** containing all the arguments. A natural use of this is when one wants to apply a function like `cbind` to all elements of a list or data frame.* »

```
is.na.data.frame <- function (x) {
  y <- do.call(cbind, lapply(x, is.na))
  rownames(y) <- row.names(x)
  y
}
```

CONTRÔLE — LIRE CETTE FONCTION.

`lapply(x, is.na)` rend **une liste** de vecteurs logiques, un par colonne (fiche 306). `cbind` accepte un nombre **quelconque** d'arguments, mais **pas une liste** : `cbind(liste)` ne ferait pas ce qu'on veut. `do.call` **déplie la liste en arguments séparés** — c'est exactement ce dont on a besoin.

« Other uses include variations over constructions like `do.call("f", list(...))`. »

● Concept 7 — Disséquer une fonction

DÉFINITION (§6.6).

« *It is often useful to be able to manipulate the components of a function or closure. R provides a set of interface functions for this purpose.* »

Fonction	Ce qu'elle rend / fait
<code>body</code>	« Returns the expression that is the body of the function »
<code>formals</code>	« Returns a list of the formal arguments. This is a pairlist. »
<code>environment</code>	« Returns the environment associated with the function »
<code>body<-</code>	« sets the body of the function to the supplied expression »
<code>formals<-</code>	« Sets the formal arguments ... to the supplied list »
<code>environment<-</code>	« Sets the environment of the function to the specified environment »

Ce sont les **trois composantes d'une fermeture** de la fiche 310 — « *a formal argument list, a body and an environment* » — chacune **lisible et modifiable**.

COMPLÉMENT (§6.6).

« It is also possible to alter the bindings of different variables in the environment of the function, using code along the lines of `evalq(x <- 5, environment(f))` . »

« It is also possible to convert a function to a list using `as.list` . The result is the concatenation of the list of formal arguments with the function body. Conversely such a list can be converted to a function using `as.function` . This functionality is mainly included for S compatibility. Notice that environment information is lost when `as.list` is used, whereas `as.function` has an argument that allows the environment to be set. »

Énoncé. Écrire `moncarre(v)` qui trace `v^2` contre `v` en étiquetant les axes avec l'expression **passée**, exactement comme le fait `plot` .

Étape 1 — le modèle à imiter. Le §6.2 donne la ligne de `plot.default` :

```
xlabel <- if (!missing(x)) deparse(substitute(x))
```

`substitute(x)` rend l'expression passée, `deparse()` en fait du texte.

Étape 2 — pourquoi cela fonctionne. « `x` **must carry information about the expression that creates its value. This is related to the lazy evaluation scheme of R. A formal argument is really a promise ...** `substitute` **will recognize a promise variable and substitute the value of its expression slot.** » La promesse (fiche 309) **conserve le texte** de ce qui a été écrit.

Étape 3 — première version, et son défaut.

```
moncarre <- function(v, x1 = deparse(substitute(v))) {  
  v <- v[!is.na(v)]          # on modifie v...  
  plot(v, v^2, xlab = x1, ylab = paste0(x1, "^2"))  
}
```

Le piège du §6.2 est exactement là : « *the rules of lazy evaluation cause the evaluation of the `yLab` expression to happen after `y` has been modified* ». La promesse `x1` ne sera forcée qu'au `plot`, **après** le filtrage — et `deparse(substitute(v))` décrirait alors `v[!is.na(v)]`, pas l'expression d'origine.

Étape 4 — la correction du cours. « **The solution is to force `yLab` to be evaluated first** » — mentionner l'argument sur une ligne seule :

```
moncarre <- function(v, x1 = deparse(substitute(v))) {  
  x1                                # FORCE la promesse MAINTENANT  
  v <- v[!is.na(v)]  
  plot(v, v^2, xlab = x1, ylab = paste0(x1, "^2"))  
}
```

Étape 5 — ce qu'il ne faut pas écrire. « **one should not use `eval(yLab)` in this situation. If `yLab` is a language or expression object, then that would cause the object to be evaluated as well, which would not at all be desirable if a math expression like `quote(log[e](y))` was being passed.** » Mentionner suffit ; évaluer détruit.

Étape 6 — vérifier le cas de l'argument manquant. La ligne de `plot.default` est protégée par `if (!missing(x))` — l'idiome de `missing()` (fiche 309), qui « **does not force evaluation of the**

argument ». Sans lui, `substitute` sur un argument absent donnerait un résultat inutilisable.

Étape 7 — *la variante mathématique*. Pour composer une étiquette **dessinée**, `bquote` est plus lisible que `substitute` :

```
ylab = bquote(.(as.name(x1))^2)
```

Et cela reste valide même si l'expression n'a **aucun sens à l'évaluation** — « *there is **no checking** that the resulting call objects make sense* » —, ce que l'annotation mathématique (fiche 314) exploite délibérément.

Étape 8 — *la leçon*. Trois mécanismes se combinent : **la promesse** garde le texte, `substitute` l'extrait, **le forçage explicite** fixe le moment. Aucun des trois n'est visible dans le code de l'appelant — c'est ce que le §6 appelle « *elegant interfaces to useful functionality* ».

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« garder une expression sans l'évaluer »	<code>quote()</code>
« récupérer ce que l'utilisateur a écrit »	<code>substitute()</code> dans une fonction
« en faire une étiquette »	<code>deparse(substitute(x))</code>
« remplacer des morceaux d'expression »	<code>substitute(e, list(...))</code> ou <code>bquote(.())</code>
« mon étiquette décrit la valeur modifiée »	forcer la promesse en la mentionnant
« évaluer chez l'appelant »	<code>eval.parent()</code>
« évaluer dans un data frame, puis chez l'appelant »	<code>eval(expr, data, sys.frame(sys.parent()))</code>
« modifier un élément d'un appel »	l'appel est une liste — <code>[[1]]</code> est la fonction
« changer la fonction appelée »	<code>appel[[1]] <- as.name("autre")</code>
« retirer un argument d'un appel »	lui assigner <code>NULL</code>
« ajouter un argument »	assignation avec étiquette
« voir comment ma fonction a été appelée »	<code>sys.call()</code> (brut) ou <code>match.call()</code> (apparié)
« ne pas transmettre les ... »	<code>match.call(expand.dots = FALSE)</code> puis <code>m\$... <- NULL</code>
« appliquer une fonction à tous les éléments d'une liste comme arguments »	<code>do.call()</code>
« le nom de la fonction est dans une variable »	<code>call()</code> ou <code>do.call("f", ...)</code>
« lire ou changer le corps d'une fonction »	<code>body()</code> , <code>body<-</code>
« changer l'environnement d'une fermeture »	<code>environment<-</code>
« comparer deux expressions »	pas par <code>deparse</code> — le dépareur n'est pas injectif

Comment résoudre ce type d'exercice

Protocole « capturer et transformer un appel » — 5 étapes.

1. `match.call()` plutôt que `sys.call()` — les arguments sont **appariés**.
2. Traiter l'objet **comme une liste** : `[[1]]` la fonction, le reste les arguments **étiquetés**.
3. **Retirer** par `NULL`, **ajouter** par assignation étiquetée, **renommer la fonction** par `as.name()`.
4. **Évaluer dans le bon cadre** — presque toujours `eval.parent()` ou `sys.frame(sys.parent())`.
5. Pour concaténer, passer par `as.list()` puis `as.call()` — « *concatenation does not work on call objects* ».

Protocole « faire parler un argument » — 4 étapes.

1. `substitute(arg)` dans la fonction — jamais à la console (au sommet, il agit comme `quote`).
2. `deparse()` pour du texte ; laisser tel quel pour une annotation mathématique.
3. **Protéger** par `if (!missing(arg))`.
4. **Forcer la promesse** avant toute modification de l'argument — **en mentionnant le nom**, jamais par `eval()`.

Protocole « construire une expression » — 3 étapes.

1. Peu de trous → `bquote()` avec `.( )`.
2. Correspondances nombreuses ou calculées → `substitute(e, list(...))`, en `quote` **ant** les symboles à insérer.
3. Se rappeler que la construction est **purement lexicale** : rien ne vérifie que le résultat a un sens.

● Common mistakes

Erreur	Correction
Croire que <code>2 + 2</code> n'est pas un appel	c'est <code>`+`(2, 2)</code>
Croire <code>[[1]]</code> être le premier argument	c'est la fonction
Utiliser <code>substitute</code> à la console	au sommet, il agit comme <code>quote</code>
Oublier <code>quote()</code> dans la liste de substitution	on insère la valeur , pas le symbole
Attendre de <code>substitute</code> une vérification	« <i>purely lexical ... no checking</i> »
Modifier l'argument avant de forcer l'étiquette	l'étiquette décrira la valeur modifiée
Forcer par <code>eval(ylab)</code>	détruit une expression mathématique
Comparer des expressions par <code>deparse</code>	le dépareur n'est pas injectif
Croire un objet <code>expression</code> et un <code>call</code> interchangeables	l'un résiste à l'évaluation, l'autre non
Se fier à l'affichage pour reconnaître un objet de langage	<code>e</code> et <code>ee</code> s'affichent pareil
Utiliser <code>sys.call()</code> pour transformer un appel	il n'apparie pas les arguments
Concaténer des objets <code>call</code> directement	passer par <code>as.list</code> puis <code>as.call</code>
Croire que <code>call("round", x)</code> insère le symbole	il insère la valeur
Passer une liste à <code>cbind</code>	il faut <code>do.call(cbind, liste)</code>
Croire <code>...</code> être un appel à <code>list</code>	c'est une pairlist — « <i>not a call to <code>list</code> like it is in S</i> »
Évaluer un appel construit dans le mauvais cadre	les variables n'ont de sens que chez l'appelant
Croire qu'évaluer dans un environnement est sans effet	« <i>may actually change that environment</i> »
Utiliser <code>as.list</code> / <code>as.function</code> par confort	« <i>mainly included for S compatibility</i> », et l'environnement est perdu

Ultimate Review

Le principe. « *subroutines have the ability to modify or construct other subroutines and evaluate the result as an integral part of the language itself* » — « *similar to Lisp and Scheme ... the engine is really very Lisp-like* ». Applications : **dérivées analytiques**, **génération de fonctions polynomiales**, `model.frame` dans les routines de modélisation, `curve` .

Trois objets de langage. **calls, expressions, functions.** `quote()` capture : « *The arguments are not evaluated, the result is simply the parsed argument.* » Et `2 + 2` est ``+(2, 2)` .

Un appel est une liste. `e[[1]]` la fonction, `e[[2]]` , `e[[3]]` les arguments ; les mots-clés servent d'étiquettes (`e3$x`) · `as.list` / `as.call` convertissent · composants de mode `"name"` , ou **constantes**, ou **autres appels** (sous-expressions) · `as.name("+")` fabrique un symbole.

Parenthèses et dépareillage. « *All grouping parentheses in input are preserved ... represented as a function call with one argument* » ; `4 - (2 - 2)` devient `"-(4, ("-(2, 2))"` · « *R's parser is not perfectly invertible, nor is its deparse* » — `deparse(quote(c(1,2)))` et `deparse(c(1,2))` donnent la même chaîne · mais « *deparsed expressions should evaluate to an equivalent value* ».

`substitute()` . L'usage type : `deparse(substitute(x))` au début de `plot.default` · fonctionne parce qu'« *a formal argument is really a promise* » à **trois fentes** et que « *substitute will recognize a promise variable and substitute the value of its expression slot* » · **les variables locales sont aussi substituées** · deuxième argument : **une liste étiquetée ou un environnement** ; `quote()` nécessaire pour insérer un symbole.

Les règles exactes. Objet local simple → **sa valeur**, « *except if matching against the global environment* » · promesse → **son expression** · non apparié → **inchangé**. L'exception du sommet est « *admittedly peculiar ... inherited from S* » : au niveau global, `substitute` agit comme `quote` .

Purement lexical. « *the substitutions are purely lexical ; there is no checking that the resulting call objects make sense* » — `substitute(x <- x + 1, list(x = 2))` rend `2 <- 2 + 1` . Et certaines parties de R **exploitent** cette liberté, comme les maths en graphique.

La double substitution. « *substitute will not evaluate its first argument* » → `eval(substitute(substitute(e, list(x = 3)), list(e = expr)))` rend `3 + y` .

Le piège de la paresse. `logplot` avec `ylab = deparse(substitute(y))` produit « *an ugly c(...) expression* » si `y` est modifié avant. **La solution : mentionner `ylab` seul.** Et « *one should not use `eval(ylab)` ... not at all be desirable if a math expression ... was being passed* ».

`bquote()` . « *The expression is quoted except for the contents of `()` subexpressions, which are replaced* » — plus compact que `substitute` sur peu de trous.

`eval()` . Par défaut « *the evaluation frame from which `eval` is called* » · `eval.parent(expr)` = `eval(expr, sys.frame(sys.parent()))` · `eval(expr, data, sys.frame(sys.parent()))` = évaluer dans une liste avec un enclos — c'est le `data =` de `lm` · « *evaluation in a given environment may actually change that environment* », mais « *the original list does not change because one is really working on a copy* ».

Objets `expression` . « *very similar to lists of call objects* » · `eval(ex)` évalue chacun, la valeur est celle du dernier · « *Expression objects ... in a sense retain their quotedness. The evaluator will not evaluate an expression object recursively, only when it is passed directly to `eval`* » · le dépareur les représente par l'appel qui les crée, d'où `e` et `ee` identiques à l'affichage, l'un de mode `"call"`, l'autre `"expression"` .

L'appel courant. `sys.call()` — tel qu'écrit, « *not really useful except for debugging* » · `match.call()` — « *with all actual arguments bound to the corresponding formals* ».

Le patron de `lm` . `match.call()` → `NULL` pour retirer → **assignation étiquetée** pour ajouter → `[[1]]`
`<- as.name("model.frame")` pour renommer → `eval(mf, sys.frame(sys.parent()))` .

`expand.dots = FALSE` . « *lets all ... arguments be collected as a single argument with the tag ...* » · « *The ... argument is a list (a pairlist to be precise), not a call to `List` like it is in S* » · usages : supprimer les ... (`plot.formula`) et fusionner des arguments (`update.default`) · « *Concatenation does not work on call objects without the coercion* » — `as.list` puis `as.call` .

Construire. `call("round", x)` — « *the value of `x` rather than the symbol is inserted* », utile « *where the name of a function is available as a character variable* » · `do.call(f, liste)` — « *evaluates the call immediately and takes the arguments from an object of mode "List"* » ; usage naturel : `do.call(cbind, lapply(x, is.na))` .

Disséquer une fonction. `body` , `formals` (« *This is a pairlist* »), `environment` — et leurs versions de remplacement · `evalq(x <- 5, environment(f))` pour changer une liaison · `as.list` / `as.function` : « *mainly included for S compatibility* », et `as.list` perd l'environnement.

Active Recall

« *R belongs to a class of programming languages in which subroutines have the ability to modify or construct other subroutines and evaluate the result as an integral part of*

the language itself. This is similar to Lisp and Scheme ... but in contrast to FORTRAN and the ALGOL family. » (§6)

« R presents a friendlier interface to programming than Lisp does ... but the engine is really very Lisp-like. R allows direct access to parsed expressions and functions and allows you to alter and subsequently execute them, or create entirely new functions from scratch. »

Les usages cités : dérivées analytiques, génération de fonctions polynomiales, les appels « *admittedly not very pretty* » à `model.frame` , et `curve` — « *elegant interfaces to useful functionality* ».

« The most direct method of obtaining a call object is to use `quote` ... The arguments are not evaluated, the result is simply the parsed argument. » (§6.1)

```
e1 <- quote(2 + 2)
e2 <- quote(plot(x, y))
```

« It is perhaps most immediately obvious why the `e2` object has mode `"call"` ... However, `e1` actually has exactly the same structure as a call to the binary operator `+` with two arguments » :

```
> quote("+"(2, 2))
2 + 2
```

C'est la règle du §3.1.4 (fiche 301) : « *except for the syntax, there is no difference between applying an operator and calling a function* ». Tout est appel.

« The components of a call object are accessed using a list-like syntax, and may in fact be converted to and from lists using `as.list` and `as.call` . » (§6.1)

```
> e2[[1]] # plot    <- LA FONCTION
> e2[[2]] # x
> e2[[3]] # y
```

« *When keyword argument matching is used, the keywords can be used as list tags* » :

```
e3$x rend age pour e3 <- quote(plot(x = age, y = weight)) .
```

Les composants sont de mode `"name"` pour les identifiants, mais peuvent aussi être **des constantes** ou **d'autres appels** — « *subexpressions are simply components that are themselves calls* ».

Et `as.name("+")` fabrique un symbole : `e2[[1]] <- as.name("+")` transforme `plot(x, y)` en `x + y` .

« *As it happens, R's parser is not perfectly invertible, nor is its deparse* » (§6.1) :

```
> str(quote(c(1,2)))      # language c(1, 2)
> str(c(1,2))             # num [1:2] 1 2
> deparse(quote(c(1,2)))  # "c(1, 2)"
> deparse(c(1,2))        # "c(1, 2)"
```

Deux objets de nature totalement différente — un appel et un vecteur numérique — ont **le même dépareillage**. De même, `quote("-"(2, 2))` et `quote(2 - 2)` s'affichent tous deux `2 - 2` .

« *Deparsed expressions should, however, evaluate to an equivalent value to the original expression (up to rounding error)*. » — le dépareillage préserve **le sens**, pas **l'identité**.

Pour comparer, utiliser `identical()` .

Même leçon au §6.4 : `e <- quote(expression(2+2))` et `ee <- expression(2+2)` « **look identical when printed** » mais sont de modes différents.

La ligne du début de `plot.default` (§6.2) :

```
xlabel <- if (!missing(x)) deparse(substitute(x))
```

« *This causes the variable or expression given as the `x` argument to `plot` to be used for labeling the x-axis.* »

Le mécanisme est la promesse : « *for this to happen, `x` must carry information about the expression that creates its value. This is related to the lazy evaluation scheme of R.* »

A formal argument is really a promise, an object with three slots — l'expression, l'environnement, la valeur. `substitute` **will recognize a promise variable and substitute the value of its expression slot.** »

La fonction ne reçoit pas le nom « poids » — elle le retrouve dans la fente expression de la promesse (fiche 309).

Et « **if `substitute` is invoked inside a function, the local variables of the function are also subject to substitution** ».

« **It is important to realize that the substitutions are purely lexical ; there is no checking that the resulting call objects make sense if they are evaluated.** `substitute(x <- x + 1, list(x = 2))` **will happily return** `2 <- 2 + 1 .` » (§6.2)

`substitute` **fabrique de la syntaxe, pas du sens.** Le résultat peut être parfaitement inévaluable.

Et ce n'est pas un défaut : « **some parts of R make up their own rules for what makes sense and what does not and might actually have a use for such ill-formed expressions.** For example, using the "math in graphs" feature often involves constructions that are syntactically correct, but which would be meaningless to evaluate, like `{ }>=40*" years" .` »

Une expression destinée à être **dessinée** (fiche 314) n'a aucune raison d'être évaluable.

```
logplot <- function(y, ylab = deparse(substitute(y))) {  
  y <- log(y)  
  plot(y, ylab = ylab)  
}
```

« **This looks straightforward, but one will discover that the y label becomes an ugly `c(...)` expression.** It happens because the rules of lazy evaluation cause the

evaluation of the `yLab` expression to happen after `y` has been modified. » (§6.2)

La promesse de `yLab` n'est forcée qu'au `plot` — après `y <- log(y)` . `substitute(y)` décrit alors la **nouvelle** valeur.

La correction : « *force `yLab` to be evaluated first* » — le mentionner sur une ligne seule.

Et pas par `eval` : « *one should not use `eval(yLab)` in this situation. If `yLab` is a language or expression object, then that would cause the object to be evaluated as well, which would not at all be desirable if a math expression like `quote(Log[e](y))` was being passed.* »

« *A variant on `substitute` is `bquote`, which is used to replace some subexpressions with their values.* » (§6.2)

	Écriture
<code>substitute</code>	<code>substitute(x[ix] == y, list(ix = i, y = pnorm(i)))</code>
<code>bquote</code>	<code>bquote(x[.(i)] == .(pnorm(i)))</code>

« *The expression is quoted except for the contents of `.()` subexpressions, which are replaced.* »

La différence est la lisibilité : `substitute` demande **une liste de correspondances** séparée de l'expression ; `bquote` **marque les trous là où ils sont**. Sur deux ou trois substitutions, `bquote` est nettement plus clair — c'est d'ailleurs ainsi que le cours réécrit son propre exemple, « *more compactly* ».

« *It is also possible to specify the environment in which the evaluation is to take place. By default this is the evaluation frame from which `eval` is called, but quite frequently it needs to be set to something else.* » (§6.3)

Cas 1 — chez l'appelant. « *when the object to evaluate is the result of a `substitute` operation ... it will contain variables that make sense to the caller only* (notice that **there**

is no reason to expect that the variables of the caller are in the lexical scope of the callee) ». D'où `eval.parent(expr)` , raccourci de `eval(expr, sys.frame(sys.parent()))` .

Cas 2 — dans une liste, avec enclos. `eval(expr, data, sys.frame(sys.parent()))` : « *one needs not only to evaluate an expression in a list, but also to specify an enclosure into which the search continues if the variable is not in the list* ».

C'est exactement le `data =` de `lm` (fiche 313) : d'abord le data frame, ensuite l'appelant.

« Objects of mode `"expression"` ... are **very similar to lists of call objects** ... *evaluating an expression object evaluates each call in turn, but the final value is that of the last call* » (§6.4).

La différence subtile : « *Call objects are indistinguishable from subexpressions in a parse tree. This means that they are automatically evaluated in the same way a subexpression would be. Expression objects can be recognized during evaluation and in a sense retain their quotedness. The evaluator will not evaluate an expression object recursively, only when it is passed directly to `eval` .* »

```
> eval(substitute(mode(x), list(x = quote(2 + 2))))      # "numeric"
> eval(substitute(mode(x), list(x = expression(2 + 2)))) # "expression"
```

Le `call` s'est évalué en chemin ; l'objet `expression` a résisté.

```
> f <- function(x, y, ...) sys.call()
> f(y = 1, 2, z = 3, 4)
f(y = 1, 2, z = 3, 4)

> f <- function(x, y, ...) match.call()
> f(y = 1, 2, z = 3, 4)
f(x = 2, y = 1, z = 3, 4)
```

« `sys.call` ... is not really useful except for debugging because it requires the function to keep track of argument matching in order to interpret the call. For instance, it must be able to see that the 2nd actual argument gets matched to the first formal one. »

« More often one requires the call with all actual arguments bound to the corresponding formals. To this end, the function `match.call` is used. » (§6.5)

`match.call()` a fait le travail d'appariement (fiche 309, trois passes) : le 2 est maintenant étiqueté `x =` et placé en tête.

```
mf <- cl <- match.call()
mf$singular.ok <- mf$model <- mf$method <- NULL
mf$drop.unused.levels <- TRUE
mf[[1]] <- as.name("model.frame")
mf <- eval(mf, sys.frame(sys.parent()))
```

« The call can be treated as a list object where the first element is the name of the function and the remaining elements are the actual argument expressions, with the corresponding formal argument names as tags. » (§6.5)

Opération	Technique
retirer un argument	« <i>assign</i> <code>NULL</code> »
ajouter un argument	« <i>tagged list assignment</i> »
changer la fonction	« assign to the first element ... <i>make sure that the value is a name</i> » — <code>as.name()</code> ou <code>quote()</code>
exécuter	« <i>evaluated in the parent frame, in which one can be certain that the involved expressions make sense</i> »

1m réécrit son appel en un appel à `model.frame` .

« *a switch which if set to `FALSE` lets all `...` arguments be collected as a single argument with the tag `...`* » (§6.5) :

```
> f(y = 1, 2, z = 3, 4)
f(x = 2, y = 1, ... = list(z = 3, 4))
```

« *The `...` argument is a list (a pairlist to be precise), not a call to `list` like it is in S.* »

Deux usages. (1) « *simply to get rid of any `...` arguments in order not to be passing unspecified arguments on to functions that may not know them* » — `plot.formula` fait `m$... <- NULL` . (2) `update.default` , où « *a set of optional extra arguments can **add to**, **replace**, or **cancel** those of the original call* ».

« *Concatenation does not work on call objects without the coercion ... this is arguably a bug* » — d'où `as.call(c(as.list(call), extras))` .

« *The function `call` allows creation of a call object from the function name and the list of arguments* » (§6.5) :

```
> x <- 10.5
> call("round", x)
round(10.5)
```

« *the value of `x` rather than the symbol is inserted in the call, so it is distinctly different from `round(x)` ... occasionally useful where the name of a function is available as a character variable* ».

« *The function `do.call` is related, but evaluates the call immediately and takes the arguments from an object of mode `"list"` containing all the arguments.* »

```
y <- do.call(cbind, lapply(x, is.na))
```

`lapply` rend une liste ; `cbind` veut des arguments séparés ; `do.call` déplie la liste en arguments. « A natural use of this is when one wants to **apply a function like `cbind` to all elements of a list or data frame** ».

« *It is often useful to be able to manipulate the components of a function or closure. R provides a set of interface functions for this purpose.* » (§6.6)

Lire	Écrire
<code>body(f)</code> — « the expression that is the body »	<code>body(f) <-</code>
<code>formals(f)</code> — « a list ... This is a pairlist »	<code>formals(f) <-</code>
<code>environment(f)</code>	<code>environment(f) <-</code>

Ce sont les **trois composantes d'une fermeture** (fiche 310).

*« It is also possible to **alter the bindings of different variables in the environment of the function**, using `evalq(x <- 5, environment(f))` . »*

Sur `as.list` / `as.function` : « **This functionality is mainly included for S compatibility. Notice that environment information is lost when `as.list` is used**, whereas `as.function` has an argument that allows the environment to be set. »

Flashcards

Question	Réponse
À quelle famille de langages R se rattache-t-il ici ?	Lisp et Scheme
Que peut faire un sous-programme en R ?	Modifier ou construire d'autres sous-programmes
Comment le manuel qualifie-t-il le moteur ?	« <i>very Lisp-like</i> »
Deux applications standard citées ?	Dérivées analytiques, génération de polynômes
Une fonction qui prend une expression ?	<code>curve</code>
Les trois sortes d'objets de langage ?	Calls, expressions, fonctions
Comment capturer une expression ?	<code>quote()</code>
Ses arguments sont-ils évalués ?	Non
Que vaut <code>quote("+"(2, 2))</code> ?	<code>2 + 2</code>
Que contient <code>e[[1]]</code> d'un appel ?	La fonction
Et <code>e[[2]]</code> ?	Le premier argument
Comment convertir un appel en liste ?	<code>as.list</code> — et retour par <code>as.call</code>
Comment accéder à un argument nommé ?	Par <code>\$</code> , le mot-clé sert d' étiquette
Le mode des identifiants dans un appel ?	<code>"name"</code>
Comment fabriquer un symbole ?	<code>as.name("...")</code>
Que deviennent les parenthèses de groupement ?	Conservées , comme un appel à un argument
Que fait l'opérateur <code>()</code> à l'évaluation ?	Il rend son argument
Le parseur est-il inversible ?	Non — ni le dépareur
Que valent <code>deparse(quote(c(1,2)))</code> et <code>deparse(c(1,2))</code> ?	La même chaîne
Que garantit le dépareillage ?	Une valeur équivalente à l'évaluation
Comment récupérer l'expression d'un argument ?	<code>substitute(arg)</code>

Question	Réponse
Comment en faire du texte ?	<code>deparse(substitute(arg))</code>
Où voit-on cet idiome ?	Au début de <code>plot.default</code>
Pourquoi cela fonctionne-t-il ?	L'argument est une promesse
Que lit <code>substitute</code> dans la promesse ?	Sa fente expression
Les variables locales sont-elles substituées ?	Oui
Le second argument de <code>substitute</code> ?	Une liste étiquetée ou un environnement
Pourquoi <code>quote(x)</code> dans cette liste ?	Pour insérer le symbole , pas la valeur
Les substitutions vérifient-elles le sens ?	Non — « <i>purely lexical</i> »
Que rend <code>substitute(x <- x+1, list(x=2))</code> ?	<code>2 <- 2 + 1</code>
Qui exploite cette liberté ?	Les maths en graphique
<code>substitute</code> évalue-t-il son 1 ^{er} argument ?	Non — d'où la double substitution
Que fait <code>substitute</code> au niveau global ?	Il agit comme <code>quote</code>
D'où vient cette exception ?	De S — « <i>admittedly peculiar</i> »
Le piège de <code>logplot</code> ?	L'étiquette décrit la valeur modifiée
La solution ?	Mentionner l'argument sur une ligne seule
Pourquoi pas <code>eval(ylab)</code> ?	Cela évaluerait une expression mathématique
La variante compacte de <code>substitute</code> ?	<code>bquote()</code>
Que marque <code>.()</code> ?	Le sous-terme à remplacer
Où <code>eval</code> évalue-t-il par défaut ?	Dans le cadre appelant <code>eval</code>
Le raccourci pour le cadre parent ?	<code>eval.parent()</code>
Son équivalent complet ?	<code>eval(expr, sys.frame(sys.parent()))</code>
Comment évaluer dans un data frame ?	<code>eval(expr, data, enclos)</code>

Question	Réponse
À quoi sert l'enclos ?	La recherche continue si la variable manque
Évaluer peut-il modifier l'environnement ?	Oui
Et une liste ?	Non — on travaille sur une copie
Que vaut <code>eval(expression(2+2, 3+4))</code> ?	7 — la valeur du dernier
Qu'ont de particulier les objets <code>expression</code> ?	Ils conservent leur quotation
Quand sont-ils évalués ?	Seulement passés directement à <code>eval</code>
Comment le dépareur les représente-t-il ?	Par l'appel qui les crée
Que rend <code>sys.call()</code> ?	L'appel tel qu'écrit
Est-ce utile ?	Surtout pour déboguer
Que rend <code>match.call()</code> ?	L'appel avec arguments appariés
Comment retirer un argument d'un appel ?	Lui assigner <code>NULL</code>
Comment en ajouter un ?	Assignment étiquetée
Comment changer la fonction appelée ?	<code>appel[[1]] <- as.name("...")</code>
Où évaluer l'appel reconstruit ?	Dans le cadre parent
Que fait <code>expand.dots = FALSE</code> ?	Regroupe les <code>...</code> en un seul argument
Quelle est la nature de <code>...</code> ?	Une pairlist — pas un appel à <code>list</code>
Comment concaténer des appels ?	<code>as.list</code> puis <code>as.call</code>
Que fait <code>call("round", x)</code> ?	Crée l'appel avec la valeur de <code>x</code>
Quand est-ce utile ?	Quand le nom de la fonction est une chaîne
Que fait <code>do.call</code> ?	Il construit et évalue aussitôt
D'où prend-il les arguments ?	D'une liste
Son usage naturel ?	<code>do.call(cbind, lapply(x, is.na))</code>

Question	Réponse
Lire le corps d'une fonction ?	<code>body()</code>
Ses arguments formels ?	<code>formals()</code> — une pairlist
Son environnement ?	<code>environment()</code>
Sont-ils modifiables ?	Oui , par les versions <code><-</code>
Changer une liaison dans son environnement ?	<code>evalq(x <- 5, environment(f))</code>
Pourquoi <code>as.list</code> / <code>as.function</code> existent-elles ?	Pour la compatibilité S
Que perd <code>as.list</code> ?	L' environnement